RustReader Deep Scan ??? Technical Report

RustReader Deep Scan — Technical Report

OpenCV AI Competition 2026 · Agentic Vision path · Team: RustReader (R. Subramanian, solo)

1. Problem

Old fasteners rust. Rust and protective coatings hide head markings, and corroded threads are hard to read by eye. Mechanics and workshop staff (appliance, car and lorry repair; small manufacturing units) then guess the "closest fit" or travel to hardware shops carrying the old part. In one case we documented, a mechanic could not identify a battery bolt worth about ₹30; the alternative was a ₹5,000 replacement assembly. Existing fastener-ID apps are trained on clean catalogue images and fail on exactly these rusted parts.

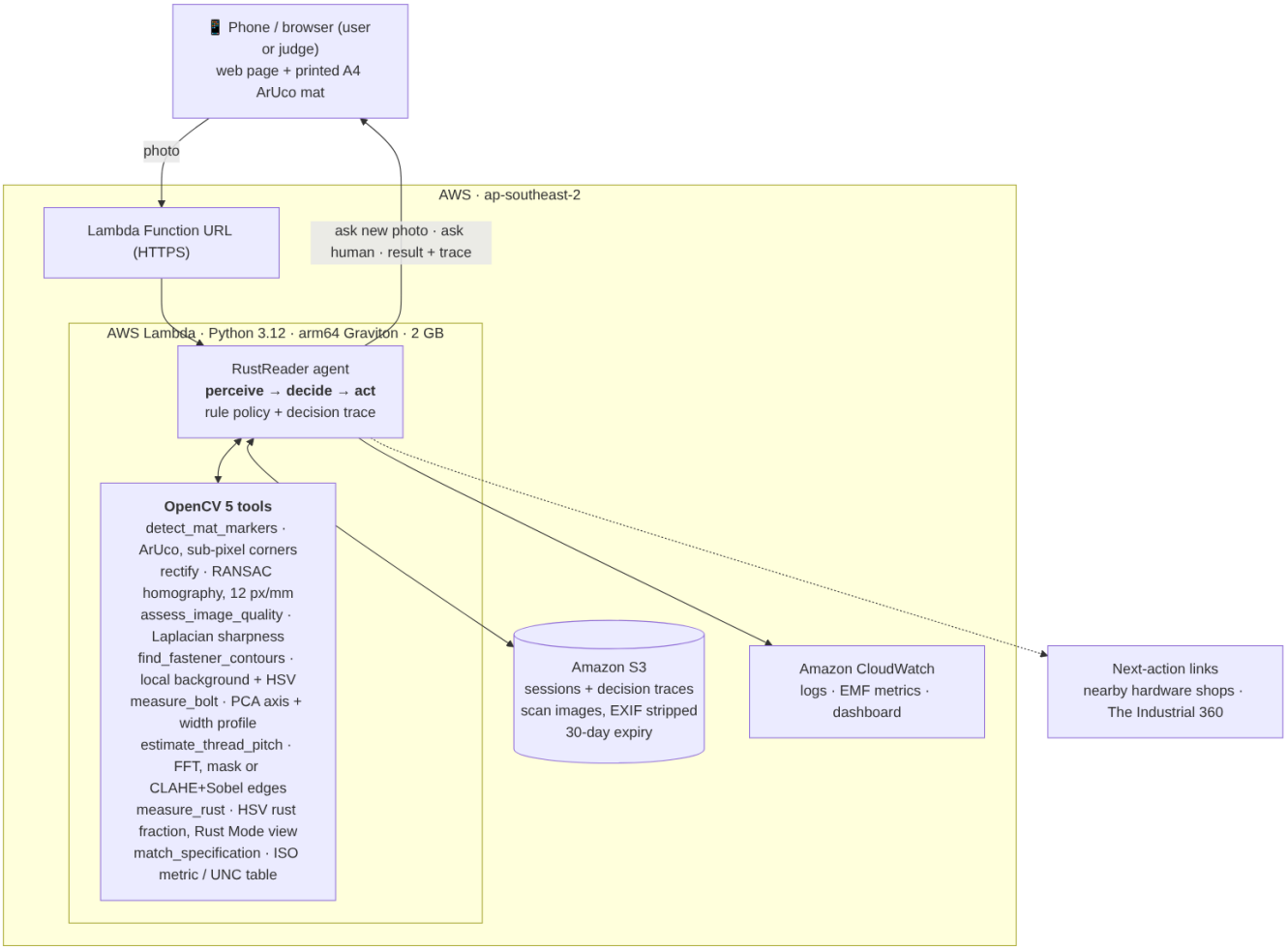
2. Users and impact

- **Mechanics and workshops**: identify size, pitch and length without a full gauge set.
- **Hardware shops / industrial suppliers**: match a customer's rusted sample to stock.
- **DIY users**: know what to ask for before travelling.

The result is actionable: an exact part name (e.g. `M12x1.25 x 50 hex bolt`), spanner size, a rust-based replace/reuse recommendation, and links to find the part nearby or order online.

3. System overview

A phone/browser page sends the photo to an AWS Lambda function (arm64 Graviton) that runs an agent over OpenCV 5 tools. Sessions, traces and scans are stored in Amazon S3; metrics and logs go to CloudWatch. See Figure 1 (architecture) and Figure 2 (agent workflow).



RustReader also has an offline Flutter app (on-device TFLite classification). This cloud "deep scan" adds metric measurement, thread pitch and the agent loop, which need more compute and a calibration reference.

4. OpenCV 5 implementation

Tool (file)	OpenCV 5 functions	Purpose
`detect_mat_markers` (vision/marker_detection.py)	`aruco.ArucoDetector`, `CORNER_REFINE_SUBPIX`	find the 4 mat markers with sub-pixel corners
`compute_photo_to_mat_homography`	`findHomography` (RANSAC), `perspectiveTransform`	perspective + scale; reprojection error as a quality signal
`rectify_photo_to_mat`	`warpPerspective` (cubic)	top-down image at exactly 12 px/mm
`assess_image_quality` (vision/image_quality.py)	`Laplacian` variance on rectified markers	scale-independent blur check; paper brightness
`find_fastener_contours` (vision/segmentation.py)	`resize` + `medianBlur` background model, `subtract`, `cvtColor` HSV, `morphologyEx`, `findContours`	segment dark **or** rust-coloured objects robustly under uneven light
`measure_bolt` (vision/measurement.py)	`PCACompute`, `getRotationMatrix2D`, `warpAffine`, `drawContours`	align bolt, column-wise width profile → head end, head type (hex / socket / countersunk / flanged), major/minor diameter, nominal length (countersunk includes head), head width

Tool (file)	OpenCV 5 functions	Purpose
`estimate_thread_pitch` (vision/thread_pitch.py)	`createCLAHE`, `bilateralFilter`, `Sobel` (+ NumPy FFT)	thread edge signal → detrend → threaded section → Hann-windowed FFT → pitch; two edge modes: binary mask, or CLAHE+Sobel with sub-pixel peak
`measure_rust` / Rust Mode (vision/rust_analysis.py)	HSV thresholds, LAB `CLAHE`, unsharp mask (`GaussianBlur`, `addWeighted`)	rust fraction → severity; enhanced view for worn markings
`match_specification` (vision/spec_matching.py)	—	weighted error vs ISO 261 / ISO 8676 / ASME B18.2.1 table; hex seen across flats **or** corners

Key design choices:

- **Measurement by geometry, not learning.** Size is physics: a known mat gives millimetres directly. This needs no training data and stays correct on rust the model never saw.
- **Thread pitch by frequency.** Rust adds noise but the thread repeats; the FFT peak is a robust estimate, and its peak-to-median ratio is a usable confidence.
- **Lighting-tolerant segmentation.** A per-pixel paper model (large median filter at 1/8 scale) handles shadows and vignetting better than a single threshold.

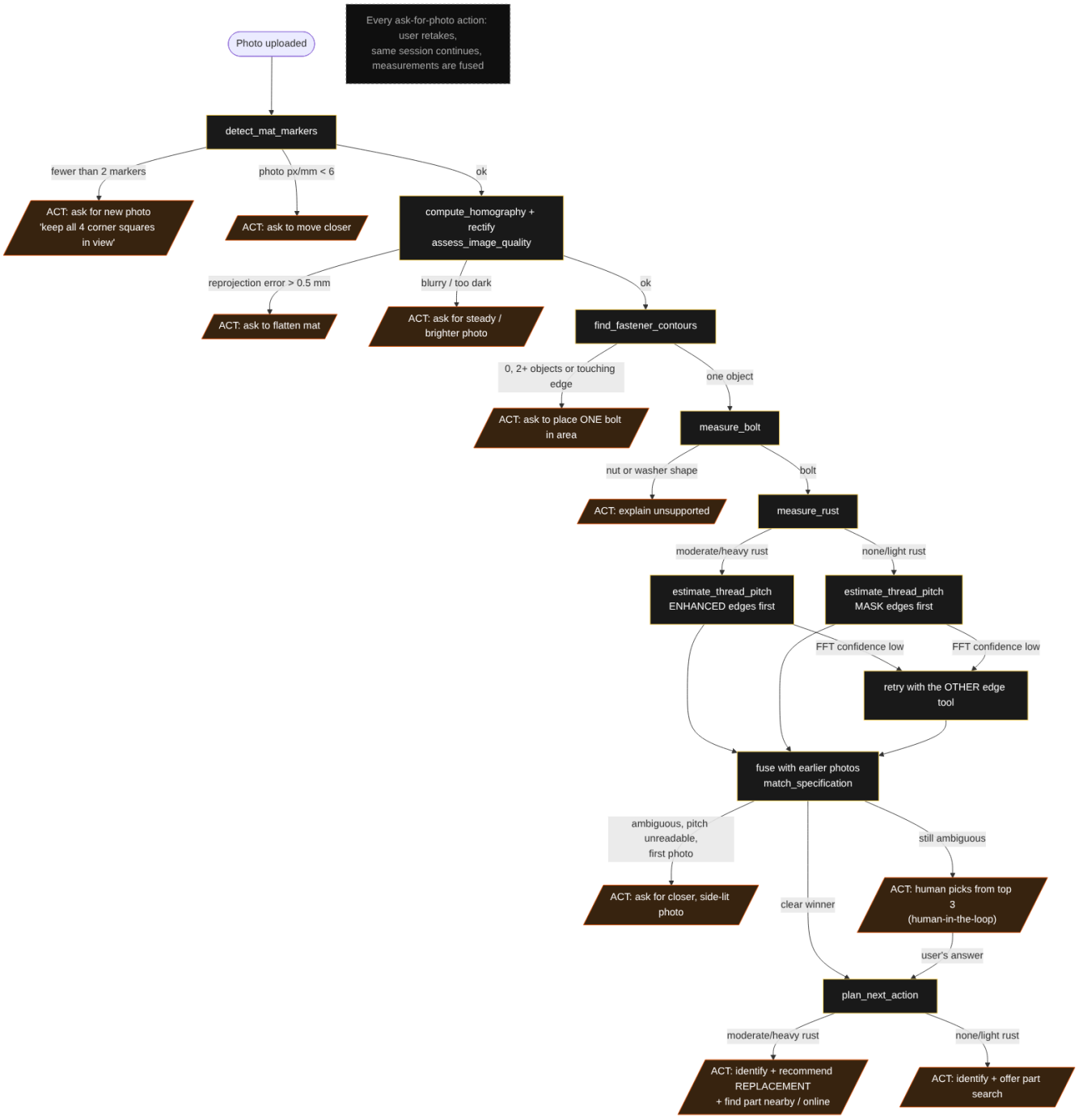
5. Agent: perception → decision → action

The agent (`agent/rustreader_agent.py`) runs the tools in order, but **observations change later tool calls and actions**: | Observation | Decision / action | |---|---| | < 2 markers, or photo < 6 px/mm | ask for a new photo (reason-specific instruction) | | homography reprojection error > 0.5 mm | ask user to flatten the mat | | marker sharpness < 120 / paper too dark | ask for a steadier / brighter photo | | 0 objects, 2+ objects, or object touches the border | ask to place ONE bolt in the work area | | nut/washer shape | stop, explain unsupported | | rust moderate/heavy | run pitch with **enhanced (CLAHE+Sobel)** edges first, else mask edges | | pitch FFT confidence < 12 | **re-run pitch with the other edge tool** | | top candidates differ only in pitch and pitch unreadable (first photo) | ask for a closer, side-lit photo; measurements from both photos are fused | | still ambiguous | **human-in-the-loop**: user picks from top 3 (or "none") | | clear winner | identify; heavy/moderate rust → recommend replacement + find part; else offer part search |

Every step is appended to a trace (`tool`, `observation`, `decision`, `reason`, timestamp), stored in S3 with the session and shown in the web UI. Example (heavy-rust sample):

```
 1 detect_mat_markers     -> continue        enough markers to compute scale and perspective
2 assess_image_quality   -> continue        sharp and bright enough to measure
3 find_fastener_contours -> continue        exactly one fastener
4 measure_bolt           -> continue        bolt with shank about 11.75 mm
5 measure_rust           -> choose_pitch_tool
6 estimate_thread_pitch  -> accept_pitch    rust is heavy, so started with the enhanced edge
tool
7 match_specification    -> identify        M12x1.25 is clearly the best fit
8 plan_next_action       -> recommend_replacement_and_find_part
```

Failure handling: every tool failure maps to a specific user instruction rather than a wrong answer; the session continues with the next photo. Human control: ambiguous results are never auto-resolved, and the safety advice is fixed text, not generated.



6. AWS deployment

- **AWS Lambda**, Python 3.12, **arm64 (Graviton)**, 2048 MB, 30 s timeout; OpenCV 5.0.0 (`opencv-python-headless==5.0.0.93`) and NumPy 2.2.6 bundled as a 58 MB zip (146 MB unzipped).
- **Lambda Function URL** (HTTPS) serves the web page, `/analyze`, `/answer`, `/session/<id>`, the printable mat and demo samples. Public access is restricted to URL invocation.
- **Amazon S3**: sessions + traces (`sessions/`), re-encoded scans (`scans/`); public access blocked; lifecycle expiry 30 days; least-privilege role (Get/Put on this bucket only).
- **Amazon CloudWatch**: Embedded Metric Format (`RustReader/LatencyMs`, `Requests` by route and agent action) without extra libraries; dashboard with outcomes, p50/p95 latency, errors; 14-day log retention.
- Reproducible: `infra/deploy.sh` (idempotent), `infra/smoke_test.sh`, `infra/teardown.sh`; pinned dependencies in `requirements.txt` and `infra/requirements-lambda.txt`.

7. Evaluation

7.1 Synthetic bolts (exact ground truth)

`tools/make_synthetic_photo.py` renders bolts with known diameter, pitch, length, head and rust on the mat, then warps them into a perspective "phone photo" with blur and noise. `python eval/evaluate.py eval/synthetic_ground_truth.csv`:

Metric	Result
Photos	11 (8 measurable, 3 deliberate failures)
Correct thread spec when identified	8/8 (incl. fine thread M12x1.25 under heavy rust, 5/16-18 UNC, partially threaded M10x80, M6x16 countersunk, M8x25 socket head, one marker covered)
Length error	mean 0.04 mm, max 0.33 mm
Pitch error	mean 0.00 mm, max 0.01 mm
Diameter error	mean 0.28 mm, max 0.44 mm (systematic under-read, see limitations)
Failure photos (blurry, two bolts, worn threads)	3/3 correctly sent back for a new photo
Processing time	median 125 ms per photo (Mac, Arm); 761 ms round trip to Lambda in Sydney (smoke test)

7.2 Real rusted bolts

To be filled from `eval/ground_truth_results_summary.txt`: N bolts measured with a digital caliper and pitch gauge, 2 photos each, mixed rust levels.

Metric	Result
Photos / bolts	
Identified on first photo	
Correct thread spec when identified	
Diameter error (mean / max)	
Length error (mean / max)	
Pitch error (mean / max)	
Asked for new photo / asked human	
Cloud latency p50 / p95 (CloudWatch)	

8. Limitations

- Requires the printed mat (scale reference). Printing at the wrong scale gives wrong sizes; the mat has a 100 mm check ruler.
- Synthetic results are optimistic; real rust, glare and shadows are harder.
- The silhouette under-reads major diameter by about 0.3 mm on synthetic threads (thread crests on top and bottom are half a pitch out of phase); to be calibrated on the real set.
- A bolt lying flat sits ~d/2 above the paper, causing a small magnification (< 2% at normal distance).
- A side view shows the hex head across flats or across corners; both are tested, so the head is a soft cue only.

- Head type (hex / socket cap / countersunk / flanged) is classified from the side-view profile; pan, button and carriage heads are not modelled yet. Nuts and washers are detected and declined.
- Rust thresholds are colour-based and may misjudge painted or zinc-coated parts.
- The decision policy is hand-designed and auditable, not learned.

9. Responsible use

- Advisory output only; fixed safety text: a photo cannot see stretch, fatigue or cracks; torque-to-yield bolts are single-use; "when in doubt, replace".
- Human-in-the-loop on ambiguity; the agent never silently picks between close candidates.
- Privacy: images re-encoded (EXIF/GPS stripped), private bucket, 30-day expiry, no accounts or personal data collected.
- Cost control: Lambda scales to zero; no always-on servers.